



- ❖ Donner le ou les noms de briques qui permettent de faire la communication

Synchrone

Exemple de communication asynchrone : JMS (java message service) permet de d'envoyer et de recevoir de messages asynchrone entre les applications ou composants java

- ❖ -la brique JEE qui permet de faire des traitements parallèles et la classe java implémenter :

Java.util.concurrent

Class Thread

- ❖ -les principales différences entre les web services REST et les web services SOAP
 - -SOAP (Simple Object Access Protocol) est un protocole basé sur XML tandis que REST (Representation state Transform) est un style d'architecture logicielle définissant un ensemble de contraintes à utiliser pour créer des services web
- ❖ -JPA (JAVA Persistence API) : interface de programme, c'est – à-dire ensemble de fonctions, classes ou des méthodes servant de façade donnant accès à des services qui permettent d'organiser les données rationnelles
- ❖ -Java EE est une extension de la plateforme standard Java SE, principalement destinée au développement d'applications web
- ❖ -Une application Web est n'importe quel logiciel qui :
 - Utilise une technologie Web
 - Tourne dans un navigateur
 - Offre un certain nombre de fonctionnalités sur un réseau
- ❖ MVC (Modèle-vue-contrôleur)

□ Un modèle (Model) contient les données à afficher. □ Une vue (View) contient la présentation de l'interface graphique. □ Un contrôleur (Controller) contient la logique concernant les actions effectuées par l'utilisateur.

- ❖ Servlet

□ Le client envoie une requête HTTP □ Côté serveur Web (tomcat), la requête sera interceptée par le servlet qui joue le rôle du gendarme □ Le servlet aiguille la requête vers le programme responsable du traitement à effectuer □ Par rapport au modèle MVC □ Le servlet sera utilisé plutôt dans la partie Contrôleur

- ❖ une servlet est en réalité une simple classe Java, qui a la particularité de permettre le traitement de requêtes et la personnalisation de réponses. Pour faire simple, dans la très grande majorité des cas une servlet n'est rien d'autre qu'une classe capable de recevoir une requête HTTP envoyée depuis le navigateur de l'utilisateur, et de lui renvoyer une réponse HTTP.

L'interface Servlet □ Définit les méthodes représentant le cycle de vie d'une servlet.

- `public void init(ServletConfig config) throws ServletException`
- `public void service(ServletRequest rq, ServletResponse res) throws ServletException`
- `public void destroy()`
- `ServletConfig getServletConfig()` □ `public String getServletInfo()`



❖ -Principes du filtrage de servlets

- Décoder les requêtes du client et encoder les réponses pour le client
- Décrypter les requêtes du client et encrypter les réponses pour le client
- Authentifier les utilisateurs
- Filtrer le contenu d'une requête pour ne retenir que l'essentiel
- Convertir des images

Définir un filtre de servlet □ Un filtre de servlet est une classe qui implémente l'interface « `javax.servlet.Filter` » en définissant les méthodes suivantes : □ `init(FilterConfig)` : pour initialiser le filtre □ `doFilter()` : ce que doit faire le filtre □ `destroy()` : pour effectuer proprement la destruction □ Les filtres de servlets peuvent accéder au contexte des servlets et aux paramètres de déploiement par l'objet «`javax.servlet.FilterConfig`»

```
<filter>
<filter-name>Filtadmin</filter-name>
<filter-class>mr.FiltAdmin</filter-class>
</filter>
<filter-mapping>
<filter-name>Filtadmin</filter-name>
<url-pattern>/jee/admin/*</url-pattern>
</filter-mapping>
```

```
package mr;
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.Filter;
import javax.servlet.FilterChain;
import javax.servlet.FilterConfig;
import javax.servlet.ServletException;
import javax.servlet.ServletRequest;
import javax.servlet.ServletResponse;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpSession;
public class FiltAdmin implements Filter {
    public void destroy() {
        // TODO Auto-generated method stub
    }
    public void doFilter(ServletRequest request, ServletResponse
response, FilterChain chain) throws IOException, ServletException {

        HttpServletRequest httpReq = (HttpServletRequest) request;
        HttpSession session = httpReq.getSession();
```



```
        if (session.getAttribute("adminSession") !=null)
        {
            chain.doFilter(request, response);
        }
        else
        {

            PrintWriter out = response.getWriter();
            out.print(" seul administrateur est autorise ici");

        }
    }

    public void init(FilterConfig fConfig) throws ServletException {

    }

}
package mr;

import java.util.ArrayList;
import java.util.HashMap;

import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpSession;

import javax.ws.rs.GET;
import javax.ws.rs.POST;
import javax.ws.rs.Path;
import javax.ws.rs.Produces;
import javax.ws.rs.QueryParam;
import javax.ws.rs.core.Context;

@Produces("application/json")
@Path("/")
public class Conx {
    //connexion et ouvrir session
    @Context HttpServletRequest req;
    @POST
    @Path("/login")
    public String login(@QueryParam("login") String
login,@QueryParam("password") String password){

        HttpSession session = req.getSession();

        if (login.equals(Administrateur.login)
password.equals(Administrateur.password) ){
            session.setAttribute("adminSession",login);

            return "Bienvenue Administrateur";
        }
    }
}
```



```
}

else{session.invalidate();
    return "login ou password incorrecte";
}

}

@Path("/logout")
@GET
public String logout(){
    HttpSession session = req.getSession();
    session.invalidate();

    return "Deconnecter ";
}

}

<filter> <filter-name>SessionFilter</filter-name>
<filter-class>servlet.filtre.SessionFilter</filter-class> <init-param>
<param-name>NomAtribut</param-name>
<param-value>NbDeVisites</param-value> </init-param>
</filter>
<filter-mapping> <filter-name>SessionFilter</filter-name> <url-pattern>/*</url-pattern>
</filter-mapping>

package web.projets.bob.cryptage; import javax.servlet.*;
import javax.servlet.http.*; import java.projects.cryptage.;
public class cryptageFilter implements Filter { private String typeCodage = null;
public void init(FilterConfig filterConfig) {
typeCodage = filterConfig.getInitParameter("Codage"); }
public void doFilter(ServletRequest request,
ServletResponse response, FilterChain chain)
throws IOException, ServletException {
codage = new Codage();
codage.decode(request, typeCodage); // crypte la requête chain.doFilter(request, response); //
passe à la servlet codage.encode(response, typeCodage); // encrypte la réponse
}}
```

❖ Git

- commande git pour créer un repository git : git init ,git add
- la commande permettent de se positionner sur une commit donner : git log
- après avoir créer un fichier quels sont les commandes obligatoire pour pousser ce fichier sur un repos local :git add ,git commit